可撤销并查集 + 线段树分治

2026.4.25 cf又没打过fengqiuqiuqiu，feiqiuiquqiu怎么把F秒了 快加训，不然省赛要打不过袁熙老师了

update：2026.4.30 在袁熙老师的指导下，在线段树分治模板题处更新了cdq分治版本，对于相同trick的题目都可以用cdq分治代替线段树分治，减少空间开销，但相对难写

可撤销并查集

模板：U504765 【模板】可撤销并查集 - 洛谷

CF Round680(Div 1) C. Team-Building：Problem - 1444C - Codeforces

ABC302 Ex-Ball Collector：Ex - Ball Collector

线段树分治

模板：P5787 【模板】线段树分治 / 二分图 - 洛谷

EDU Round22 F. Bipartite Checking：Problem - 813F - Codeforces

可撤销并查集 + 线段树分治

2025北航校赛 I. Euler：Dashboard - The 20-th Beihang University Collegiate Programming Contest (BCPC 2025) - Preliminary - Codeforces

CF Round1094(Div 1+2) F. Building Tree：Problem - F - Codeforces

---

[模板]可撤销并查集

题意：有 n 个集合，维护 3 种操作：合并 x 和 y 所在的集合；查询 x 和 y 是否在同一集合；回溯到第 v 个状态

数据范围：$n, q \le 10^6$

题解：普通并查集：通过路径压缩进行加速；可撤销并查集：需要保留每一次修改的信息，因此不能使用路径压缩。合并时只使用启发式合并（按秩合并），然后用一个栈记录合并操作，撤销操作则是将栈进行回退

```
struct DSU {
    vector<int> p, sz, stk;
    DSU(int n) {
        p.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(p.begin(), p.end(), 0);
```

```
        stk.resize(0);
    }
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    void merge(int x, int y) {
        x = find(x), y = find(y);
        if(x == y) return;
        if(sz[x] < sz[y]) swap(x, y);
        stk.push_back(y);
        p[y] = x, sz[x] += sz[y];
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
    }
    void delet() {
        if(stk.empty()) return;
        int y = stk.back();
        stk.pop_back();
        sz[p[y]] -= sz[y], p[y] = y;
    }
    void back(int t = 0) {
        while(stk.size() > t)
            delet();
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    DSU dsu(n);
    vector<int> status(m + 1);
    int now = 1;
    for(int i = 1; i <= m; i++) {
        int op, x, y, v;
        cin >> op;
        if(op == 1) {
            cin >> x >> y;
            dsu.merge(x, y);
        } else if(op == 2) {
            cin >> x >> y;
            cout << dsu.query(x, y);
        } else {
            cin >> v;
            dsu.back(status[v]);
            now = v;
```

```
        }
        status[now ++] = dsu.stk.size();
    }
}
```

Team-Building

题意：给定一个 n 个点 m 条边的无向图，没有重边和自环，一共有 k 个组，每个节点都属于其中一组，可能存在某组为空。求选出两组，它们的导出子图为二分图的方案数

数据范围：$1 \le n \le 5 \times 10^5$ ，$0 \le m \le 5 \times 10^5$ ，$2 \le k \le 5 \times 10^5$

题解：用扩展域并查集维护二分图。首先一个组内不构成二分图的组一定不合法，只考虑组内能构成二分图的组。将连接组 u 和组 v 的边一起处理，可以通过边重排序实现，将连接组 u 和组 v 的边全部加入扩展域并查集中，判断是否为二分图。处理完这组边后，将加入的边撤销

```
struct DSU {
    vector<int> p, sz, stk;
    DSU(int n) {
        p.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(p.begin(), p.end(), 0);
        stk.resize(0);
    }
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    void merge(int x, int y) {
        x = find(x), y = find(y);
        if(x == y) return;
        if(sz[x] < sz[y]) swap(x, y);
        stk.push_back(y);
        p[y] = x, sz[x] += sz[y];
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
    }
    void delet() {
        if(stk.empty()) return;
        int y = stk.back();
        stk.pop_back();
        sz[p[y]] -= sz[y], p[y] = y;
    }
    void back(int t = 0) {
```

```
        while(stk.size() > t)
            delet();
    }
};

void solve() {
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> c(n + 1);
    for(int i = 1; i <= n; i++)
        cin >> c[i];
    vector<int> st(k + 1, 0);
    vector<pii> edges(m);
    DSU dsu(n * 2);
    for(int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        if(c[u] > c[v]) swap(u, v);
        edges[i] = {u, v};
        if(c[u] == c[v]) {
            dsu.merge(u, v + n);
            dsu.merge(v, u + n);
            if(dsu.find(u) == dsu.find(u + n) || dsu.find(v) == dsu.find(v + n))
                st[c[u]] = 1;
        }
    }

    int ans = 0;
    for(int i = 1; i <= k; i++)
        if(!st[i])
            ans ++;
    ans = ans * (ans - 1ll) / 2ll;

    sort(edges.begin(), edges.end(), [&](pii a, pii b){
        if(c[a.first] != c[b.first]) return c[a.first] < c[b.first];
        return c[a.second] < c[b.second];
    });
    for(int i = 0; i < m; ) {
        int cu = c[edges[i].first], cv = c[edges[i].second];
        if(st[cu] || st[cv] || cu == cv) {
            i ++;
            continue;
        }
        int now = dsu.stk.size(), fl = 1;
        while(i < m && c[edges[i].first] == cu && c[edges[i].second] == cv) {
            int u = edges[i].first, v = edges[i].second;
```

```
            i ++;
            dsu.merge(u, v + n), dsu.merge(v, u + n);
            if(dsu.find(u) == dsu.find(u + n) || dsu.find(v) == dsu.find(v + n))
                fl = 0;
        }
        if(!fl) ans --;
        dsu.back(now);
    }
    cout << ans << '\n';
}
```

Ball Collector

题意：给定一棵 n 个节点的树，每个节点上有 A_i 和 B_i 两个球。对 $v = 2, 3, \ldots, n$ 分别求解：从节点 1 沿最短路径移动到节点 v，路径上每个点选择一个球，球能获得的最大种类数

数据范围：$2 \le n \le 2 \times 10^5$，$1 \le A_i, B_i \le n$

题解：有点典。对于路径上的点，将 A_i 与 B_i 连边，得到若干个连通块，对于一个大小为 k 的连通块，如果是一棵树，那么答案为 $k-1$，否则答案为 k，因此一个连通块的贡献是 $min(\text{点数}, \text{边数})$。在并查集上动态维护答案，从 1 号节点开始 dfs，递归结束时将加入的边撤销

```
struct DSU {
    vector<int> p, stk;
    vector<int> sznode, szedge;
    int ans;
    DSU(int n) {
        p.resize(n + 1);
        sznode.assign(n + 1, 1);
        szedge.assign(n + 1, 0);
        iota(p.begin(), p.end(), 0);
        stk.resize(0);
        ans = 0;
    }
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    void merge(int x, int y) {
        x = find(x), y = find(y);
        if(x == y) {
            stk.push_back(x);
            ans -= min(sznode[x], szedge[x]);
            szedge[x] ++;
            ans += min(sznode[x], szedge[x]);
```

```
            return;
        }
        if(sznode[x] < sznode[y]) swap(x, y);
        stk.push_back(y);
        ans -= min(sznode[x], szedge[x]) + min(sznode[y], szedge[y]);
        p[y] = x, sznode[x] += sznode[y], szedge[x] += szedge[y] + 1;
        ans += min(sznode[x], szedge[x]);
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
    }
    void delet() {
        if(stk.empty()) return;
        int y = stk.back();
        stk.pop_back();
        if(p[y] == y) {
            ans -= min(sznode[y], szedge[y]);
            szedge[y] --;
            ans += min(sznode[y], szedge[y]);
        } else {
            int x = p[y];
            ans -= min(sznode[x], szedge[x]);
            sznode[x] -= sznode[y], szedge[x] -= szedge[y] + 1, p[y] = y;
            ans += min(sznode[x], szedge[x]) + min(sznode[y], szedge[y]);
        }
    }
    void back(int t = 0) {
        while(stk.size() > t)
            delet();
    }
};

void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1), b(n + 1);
    vector<vector<int>> e(n + 1, vector<int> ());
    for(int i = 1; i <= n; i++)
        cin >> a[i] >> b[i];
    for(int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
    }

```

```
    vector<int> res(n + 1);
    DSU dsu(n);
    auto dfs = [&](auto&& self, int u, int fa) -> void {
        int now = dsu.stk.size();
        dsu.merge(a[u], b[u]);
        res[u] = dsu.ans;
        for(auto v: e[u])
            if(v != fa)
                self(self, v, u);
        dsu.back(now);
    };
    dfs(dfs, 1, -1);
    for(int i = 2; i <= n; i++)
        cout << res[i] << " \n"[i == n];
}
```

[模板]线段树分治/二分图

题意：一个 n 个节点的图，在 k 时间内有 m 条边会出现后消失，即有 m 条边形如 x, y, l, r 表示连接 x 和 y 的边在 l 时刻出现 r 时刻小时。对每个时间段判断这个图是否为二分图

数据范围：$n, k \le 10^5$ ，$m \le 2 \times 10^5$ ，$1 \le x, y \le n$ ，$0 \le l \le r \le k$

题解：本模板题旨在解决一类经典的离线动态图问题。考虑一个场景：我们需要维护一个图，支持动态地加入和删除边，并实时查询图的某些性质（例如连通性、是否为二分图等）

首先思考一个简化版的问题，如果操作并非任意删除，而是严格的撤销上一步的操作，那么可以通过可撤销数据结构来实现。然而原问题要求删除任意指定时间的边，似乎超出了可撤销模型的处理能力。线段树分治的巧妙之处就在于，通过对时间维度进行分治，将一个看似需要可追溯化的复杂问题，转化为了仅需可撤销数据结构即可解决的模型

将所有操作离线，将每条边的生命周期看作时间轴上的一个区间 $[l, r]$ ，建立一颗代表总时间 $[1, T]$ 的线段树，将每条边的存在区间 $[l, r]$ 分解并挂载到线段树上 $O(\log T)$ 个对应节点上。对线段树进行 DFS，进入节点时将该节点上挂载的所有边加入可撤销数据结构中，离开节点回溯时，将进入该节点时进行的操作进行撤销

通过这种方式，任意时刻的删除操作被自然地转化为了 DFS 回溯时的撤销操作

```
struct DSU {
    vector<int> p, sz, stk;
    DSU() {}
    DSU(int n) {
        p.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(p.begin(), p.end(), 0);
```

```
        stk.resize(0);
    }
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    void merge(int x, int y) {
        x = find(x), y = find(y);
        if(x == y) return;
        if(sz[x] < sz[y]) swap(x, y);
        stk.push_back(y);
        p[y] = x, sz[x] += sz[y];
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
    }
    void delet() {
        if(stk.empty()) return;
        int y = stk.back();
        stk.pop_back();
        sz[p[y]] -= sz[y], p[y] = y;
    }
    void back(int t = 0) {
        while(stk.size() > t)
            delet();
    }
};

struct Segment_Tree {
    struct node {
        int l, r;
        vector<pii> edges;
    };
    int n, m;
    DSU dsu;
    vector<node> tr;
    vector<int> res;
    void build(int u, int l, int r) {
        tr[u] = {l, r};
        if(l == r) return;
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
    }
    Segment_Tree(int n, int m) : n(n), m(m), dsu(m * 2) {
        tr.resize((n + 10) * 4);
        res.resize(n + 1);
        build(1, 1, n);
```

```
    }
    void add(int u, int l, int r, pii e) {
        if(tr[u].l >= l && tr[u].r <= r) {
            tr[u].edges.push_back(e);
            return;
        }
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) add(u << 1, l, r, e);
        if(r > mid) add(u << 1 | 1, l, r, e);
    }
    void dfs(int u, int fl) {
        int now = dsu.stk.size();
        for(auto [u, v]: tr[u].edges) {
            dsu.merge(u, v + m), dsu.merge(v, u + m);
            if(dsu.find(u) == dsu.find(u + m) || dsu.find(v) == dsu.find(v + m))
                fl = 0;
        }
        if(tr[u].l == tr[u].r) res[tr[u].l] = fl;
        else {
            dfs(u << 1, fl), dfs(u << 1 | 1, fl);
        }
        dsu.back(now);
    }
    void work() {
        dfs(1, 1);
        for(int i = 1; i <= n; i++)
            if(res[i]) cout << "Yes\n";
            else cout << "No\n";
    }
};

void solve() {
    int n, m, k;
    cin >> n >> m >> k;
    Segment_Tree Tr(k, n);
    for(int i = 0; i < m; i++) {
        int u, v, l, r;
        cin >> u >> v >> l >> r;
        Tr.add(1, l + 1, r, {u, v});
    }
    Tr.work();
}
```

```
struct DSU {
    vector<int> p, sz, stk;
    DSU() {}
```

```
    DSU(int n) {
        p.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(p.begin(), p.end(), 0);
        stk.resize(0);
    }
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    void merge(int x, int y) {
        x = find(x), y = find(y);
        if(x == y) return;
        if(sz[x] < sz[y]) swap(x, y);
        stk.push_back(y);
        p[y] = x, sz[x] += sz[y];
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
    }
    void delet() {
        if(stk.empty()) return;
        int y = stk.back();
        stk.pop_back();
        sz[p[y]] -= sz[y], p[y] = y;
    }
    void back(int t = 0) {
        while(stk.size() > t)
            delet();
    }
};

struct Edge {
    int l, r, u, v;
};

void work(int n, int m, vector<Edge> edges) {
    DSU dsu(m * 2);
    vector<int> res(n + 1);

    auto cdq_solve = [&](auto&& self, int l, int r, vector<Edge> e, int fl) -> void {
        if(e.empty()) {
            for(int i = l; i <= r; i++)
                res[i] = fl;
            return;
        }
```

```
        int now = dsu.stk.size();
        int mid = l + r >> 1;
        vector<Edge> temp1, temp2;
        for(int i = 0, sz = e.size(); i < sz; i++) {
            if(e[i].l <= l && e[i].r >= r) {
                dsu.merge(e[i].u, e[i].v + m);
                dsu.merge(e[i].v, e[i].u + m);
                if(dsu.query(e[i].u, e[i].u + m) || dsu.query(e[i].v, e[i].v + m))
                    fl = 0;
                continue;
            }
            if(e[i].l <= mid) temp1.push_back({e[i].l, min(e[i].r, mid), e[i].u, e[i].v});
            if(e[i].r > mid) temp2.push_back({max(e[i].l, mid + 1), e[i].r, e[i].u, e[i].v});
        }

        if(l == r) res[l] = fl;
        else {
            self(self, l, mid, temp1, fl);
            self(self, mid + 1, r, temp2, fl);
        }
        dsu.back(now);
    };

    cdq_solve(cdq_solve, 1, n, edges, 1);
    for(int i = 1; i <= n; i++)
        if(res[i]) cout << "Yes\n";
        else cout << "No\n";
}

void solve() {
    int n, m, k;
    cin >> n >> m >> k;
    vector<Edge> edges;
    for(int i = 0; i < m; i++) {
        int u, v, l, r;
        cin >> u >> v >> l >> r;
        edges.push_back({l + 1, r, u, v});
    }
    work(k, n, edges);
}
```

Bipartite Checking

题意：给定一个包含 n 个点的无向图，起初图中没有任何边。同时有 q 个操作，给出 u 和 v，如果 u 和 v 之间有一条边，则将这条边删掉，否则在 u 和 v 之间连一条边。每次操作后判断图是否为二分图

数据范围：$2 \le n, q \le 100000$

```
struct DSU {
    vector<int> p, sz, stk;
    DSU() {}
    DSU(int n) {
        p.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(p.begin(), p.end(), 0);
        stk.resize(0);
    }
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    void merge(int x, int y) {
        x = find(x), y = find(y);
        if(x == y) return;
        if(sz[x] < sz[y]) swap(x, y);
        stk.push_back(y);
        p[y] = x, sz[x] += sz[y];
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
    }
    void delet() {
        if(stk.empty()) return;
        int y = stk.back();
        stk.pop_back();
        sz[p[y]] -= sz[y], p[y] = y;
    }
    void back(int t = 0) {
        while(stk.size() > t)
            delet();
    }
};

struct Segment_Tree {
    struct node {
        int l, r;
        vector<pii> edges;
    };
    int n, m;
```

```
    DSU dsu;
    vector<node> tr;
    vector<int> res;
    void build(int u, int l, int r) {
        tr[u] = {l, r};
        if(l == r) return;
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
    }
    Segment_Tree(int n, int m) : n(n), m(m), dsu(m * 2) {
        tr.resize((n + 10) * 4);
        res.resize(n + 1);
        build(1, 1, n);
    }
    void add(int u, int l, int r, pii e) {
        if(tr[u].l >= l && tr[u].r <= r) {
            tr[u].edges.push_back(e);
            return;
        }
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) add(u << 1, l, r, e);
        if(r > mid) add(u << 1 | 1, l, r, e);
    }
    void dfs(int u, int fl) {
        int now = dsu.stk.size();
        for(auto [u, v]: tr[u].edges) {
            dsu.merge(u, v + m), dsu.merge(v, u + m);
            if(dsu.find(u) == dsu.find(u + m) || dsu.find(v) == dsu.find(v + m))
                fl = 0;
        }
        if(tr[u].l == tr[u].r) res[tr[u].l] = fl;
        else {
            dfs(u << 1, fl), dfs(u << 1 | 1, fl);
        }
        dsu.back(now);
    }
    void work() {
        dfs(1, 1);
        for(int i = 1; i <= n; i++)
            if(res[i]) cout << "YES\n";
            else cout << "NO\n";
    }
};

void solve() {
    int n, m;
```

```
    cin >> n >> m;
    map<pii,int> mp;
    Segment_Tree Tr(m, n);
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        if(u > v) swap(u, v);
        if(mp.find({u, v}) == mp.end())
            mp[{u, v}] = i;
        else {
            Tr.add(1, mp[{u, v}], i - 1, {u, v});
            mp.erase(mp.find({u, v}));
        }
    }
    for(auto [e, la]: mp)
        Tr.add(1, la, m, e);
    Tr.work();
}
```

Euler

题意：一个 n 个点 m 条边的无向图，保证没有自环和重边。有 q 次独立询问，每次询问给定 k 条边，将这 k 条边从原图中临时删除，并询问当前的图至少需要多少条不相交欧拉路径才能覆盖所有点和边（可以在点上相交，不能在边上相交）

数据范围：$1 \le n \le 10^5$，$0 \le m \le 10^5$，$1 \le q \le 10^5$，$\sum k \le 2 \times 10^5$

题解：对于一个连通块，设度数为奇数的点个数为 k，当 $k = 0$ 或 2 时可以一笔画完，当 $k \ge 4$ 时需要 $\lfloor \frac{k}{2} \rfloor$ 笔，即答案为 $max(\lfloor \frac{k}{2} \rfloor, 1)$。用线段树分治 + 可撤销并查集维护

```
struct DSU {
    vector<int> p, sz;
    vector<array<int,4>> stk;
    vector<int> d, cnt;
    int ans;
    DSU() {}
    DSU(int n) {
        p.resize(n + 1);
        sz.assign(n + 1, 1);
        d.assign(n + 1, 0), cnt.assign(n + 1, 0);
        iota(p.begin(), p.end(), 0);
        stk.resize(0);
        ans = n;
    }
```

```
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    void merge(int x, int y) {
        int px = find(x), py = find(y);
        if(px == py) {
            ans -= max(cnt[px] / 2, 1);
            if(d[x] & 1) cnt[px] --;
            if(d[y] & 1) cnt[px] --;
            d[x] ++, d[y] ++;
            if(d[x] & 1) cnt[px] ++;
            if(d[y] & 1) cnt[px] ++;
            ans += max(cnt[px] / 2, 1);
            stk.push_back({x, y, px, py});
            return;
        }
        if(sz[px] < sz[py]) swap(x, y), swap(px, py);
        ans -= max(cnt[px] / 2, 1) + max(cnt[py] / 2, 1);
        if(d[x] & 1) cnt[px] --;
        if(d[y] & 1) cnt[py] --;
        p[py] = px, sz[px] += sz[py], cnt[px] += cnt[py];
        d[x] ++, d[y] ++;
        if(d[x] & 1) cnt[px] ++;
        if(d[y] & 1) cnt[px] ++;
        ans += max(cnt[px] / 2, 1);
        stk.push_back({x, y, px, py});
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
    }
    void delet() {
        if(stk.empty()) return;
        array<int,4> t = stk.back();
        stk.pop_back();
        int x = t[0], y = t[1], px = t[2], py = t[3];
        if(p[py] == py) {
            ans -= max(cnt[px] / 2, 1);
            if(cnt[x] & 1) cnt[px] --;
            if(cnt[y] & 1) cnt[px] --;
            cnt[x] --, cnt[y] --;
            if(cnt[x] & 1) cnt[px] ++;
            if(cnt[y] & 1) cnt[px] ++;
            ans += max(cnt[px] / 2, 1);
            return;
        }
        ans -= max(cnt[px] / 2, 1);
```

```
        if(cnt[x] & 1) cnt[px] --;
        if(cnt[y] & 1) cnt[px] --;
        sz[px] -= sz[py], cnt[px] -= cnt[py], p[py] = py;
        cnt[x] --, cnt[y] --;
        if(cnt[x] & 1) cnt[px] ++;
        if(cnt[y] & 1) cnt[py] ++;
        ans += max(cnt[px] / 2, 1) + max(cnt[py] / 2, 1);
    }
    void back(int t = 0) {
        while(stk.size() > t)
            delet();
    }
};

struct Segment_Tree {
    struct node {
        int l, r;
        vector<pii> edges;
    };
    int n, m;
    DSU dsu;
    vector<node> tr;
    vector<int> res;
    void build(int u, int l, int r) {
        tr[u] = {l, r};
        if(l == r) return;
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
    }
    Segment_Tree(int n, int m) : n(n), m(m), dsu(m) {
        tr.resize((n + 10) * 4);
        res.resize(n + 1);
        build(1, 1, n);
    }
    void add(int u, int l, int r, pii e) {
        if(tr[u].l >= l && tr[u].r <= r) {
            tr[u].edges.push_back(e);
            return;
        }
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) add(u << 1, l, r, e);
        if(r > mid) add(u << 1 | 1, l, r, e);
    }
    void dfs(int u) {
        int now = dsu.stk.size();
        for(auto [u, v]: tr[u].edges)
            dsu.merge(u, v);
```

```
        if(tr[u].l == tr[u].r) res[tr[u].l] = dsu.ans;
        else dfs(u << 1), dfs(u << 1 | 1);
        dsu.back(now);
    }
    void work() {
        dfs(1);
        for(int i = 1; i <= n; i++)
            cout << res[i] << '\n';
    }
};

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    vector<pii> e(m + 1);
    vector<vector<int>> f(m + 1, vector<int>());
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        e[i] = {u, v};
    }
    for(int i = 1; i <= q; i++) {
        int k, x;
        cin >> k;
        for(int j = 0; j < k; j++) {
            cin >> x;
            f[x].push_back(i);
        }
    }

    Segment_Tree Tr(q, n);
    for(int i = 1; i <= m; i++) {
        sort(f[i].begin(), f[i].end());
        int la = 0;
        for(auto x: f[i]) {
            if(la + 1 <= x - 1) Tr.add(1, la + 1, x - 1, e[i]);
            la = x;
        }
        if(la + 1 <= q) Tr.add(1, la + 1, q, e[i]);
    }
    Tr.work();
}
```

Building Tree

题意：给定一个 n 个点 m 条边的有向图，每条边有一个边权 w。对于一条路径，路径上所有边边权的集合为 S，定义路径长度为 $mex(S)$，$dis(u,v)$ 为 u 到 v 的所有路径中的最小值

现在要构建一张有 q 个点的新图，每个点有颜色 c。在新图中增加连接顶点 u 和 v 的代价是 $dis(c_u,c_v)$，求新图的最小生成树

数据范围：$1 \le t \le 10^4$，$1 \le n,m \le 3 \cdot 10^5$，$1 \le q \le n$，$1 \le u,v \le n$，$0 \le w \le m$，$1 \le c_i \le n$

```
struct DSU {
    vector<int> p, sz, id, p0;
    vector<array<int,4>> stk;
    vector<array<int,3>> edges;
    int cnt;
    DSU(int n) {
        p.resize(n + 1), p0.resize(n + 1);
        sz.assign(n + 1, 1);
        id.assign(n + 1, -1);
        iota(p.begin(), p.end(), 0);
        iota(p0.begin(), p0.end(), 0);
        stk.resize(0), edges.resize(0);
        cnt = 0;
    }
    void update(int x) {
        if(id[x] == -1) cnt ++;
        id[x] = x;
    }
    int find(int x) {
        return x == p[x] ? x : find(p[x]);
    }
    int find0(int x) {
        return x == p0[x] ? x : p0[x] = find0(p0[x]);
    }
    void merge(int x, int y, int val) {
        x = find(x), y = find(y);
        if(x == y) return;
        if(sz[x] < sz[y]) swap(x, y);
        stk.push_back({x, y, id[x], id[y]});
        if(id[x] != -1 && id[y] != -1)
            edges.push_back({val, id[x], id[y]});
        p[y] = x, sz[x] += sz[y];
        if(id[x] == -1) id[x] = id[y];
    }
    int query(int x, int y) {
        x = find(x), y = find(y);
        return x == y;
```

```
    }
    void delet() {
        if(stk.empty()) return;
        auto t = stk.back();
        int x = t[0], y = t[1], idx = t[2], idy = t[3];
        stk.pop_back();
        sz[x] -= sz[y], p[y] = y;
        id[x] = idx, id[y] = idy;
    }
    void back(int t = 0) {
        while(stk.size() > t)
            delet();
    }
    void kruskal() {
        sort(edges.begin(), edges.end());
        int now = 0, ans = 0;
        for(auto t: edges) {
            int val = t[0], u = t[1], v = t[2];
            u = find0(u), v = find0(v);
            if(u != v) {
                ans += val, now ++;
                p0[u] = v;
            }
        }
        if(now < cnt - 1) ans = -1;
        cout << ans << '\n';
    }
};

struct Segment_Tree {
    struct node {
        int l, r;
        vector<pii> edges;
    };
    int n, m;
    DSU dsu;
    vector<node> tr;
    void build(int u, int l, int r) {
        tr[u] = {l, r};
        if(l == r) return;
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
    }
    Segment_Tree(int n, int m) : n(n), m(m), dsu(m) {
        tr.resize((n + 10) * 4);
        build(1, 0, n);
    }
```

```cpp
    void add(int u, int l, int r, pii e) {
        if(tr[u].l >= l && tr[u].r <= r) {
            tr[u].edges.push_back(e);
            return;
        }
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) add(u << 1, l, r, e);
        if(r > mid) add(u << 1 | 1, l, r, e);
    }
    void dfs(int u) {
        int now = dsu.stk.size(), val = tr[u].l;
        for(auto [u, v]: tr[u].edges)
            dsu.merge(u, v, val);
        if(tr[u].l != tr[u].r) dfs(u << 1), dfs(u << 1 | 1);
        dsu.back(now);
    }
    void update(int x) {
        dsu.update(x);
    }
    void work() {
        dfs(1);
        dsu.kruskal();
    }
};

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    Segment_Tree Tr(m + 1, n);
    for(int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if(0 <= w - 1) Tr.add(1, 0, w - 1, {u, v});
        if(w + 1 <= m + 1) Tr.add(1, w + 1, m + 1, {u, v});
    }
    for(int i = 0, x; i < q; i++) {
        cin >> x;
        Tr.update(x);
    }
    Tr.work();
}
```